JSON Query by Example

A Minimum Specification — Draft 0.1

---

1. Purpose

This specification defines a convention for querying web resources for data. It extends HTTP by defining the semantics of the QUERY method and a JSON query document format that expresses both constraints and response shape in a single document. The goal is to enable clients to retrieve precisely the data they need from any web resource that supports the convention, without abandoning the resource model, addressability, or caching properties that make the web's architecture durable.

This specification builds on the IETF QUERY method draft rather than defining a new HTTP method, giving it a foundation in an emerging standard rather than requiring invention from scratch.

This specification scopes the query document format to JSON. While the query by example model could in principle work with other document formats, JSON is the natural choice for a minimum specification given its universal adoption as the data interchange format of the modern web.

---

2. Query Document Format

A query document is a valid JSON document sent as the body of a QUERY request to a web resource. The query document expresses both the constraints that select data and the shape of the response in a single unified document. The structure of the query document mirrors the structure of the data it describes.

2.1 Field Semantics

Every field in a query document has one of three meanings determined by its value:

- **Return** — a field with value `"*"` is included in the response. The response will contain this field with its value from the resource.
- **Match** — a field with a literal value other than `"*"` constrains the result to documents where that field equals that value.
- **Absent** — a field not present in the query document is not returned in the response.

2.2 Example

The following query document finds the user with id 1, returns their name and email, returns only the city from their address where country is US, and returns the id and total of their pending orders.

```json
{
  "user": {
    "id": 1,
    "name": "*",
    "email": "*",
    "address": {
      "city": "*",
      "country": "US"
    },
    "orders": {
      "status": "pending",
      "id": "*",
      "total": "*"
    }
  }
}
```

2.3 Nested Documents

A field whose value is a JSON object specifies the shape and constraints of a nested document. The query is applied recursively to the nested document. A field whose value is `"*"` and which corresponds to an object in the resource returns the entire subtree without requiring its shape to be specified.

2.4 Collections

When a field corresponds to a collection in the resource, the query document specifies the shape and constraints of each member of the collection. The response contains all members of the collection that match the constraints, each shaped according to the query document.

---

3. Operators

Operators extend the query document for cases where a literal value is insufficient to express a constraint or retrieval intention. Operators are expressed as JSON objects with a single typographical key. A field whose value is an operator object is treated as a constraint or modifier rather than a literal match.

3.1 Comparison Operators

Operator	Meaning
">"	Greater than
">="	Greater than or equal
"<"	Less than
"<="	Less than or equal
"!="	Not equal
`"	"`

Example:

```json
{
  "order": {
    "total": { ">": 100 },
    "status": { "|": ["pending", "processing"] },
    "id": "*",
    "total": "*"
  }
}
```

3.2 Collection Operators

Operator	Meaning
"^"	Order ascending by the specified field
"v"	Order descending by the specified field
"#"	Limit — return at most n members
"@"	Offset — skip the first n members

Example:

```json
{
  "orders": {
    "^": "total",
    "#": 10,
    "@": 20,
    "status": "pending",
    "id": "*",
    "total": "*"
  }
}
```

---

4. Response Format

The response to a QUERY request is a valid JSON document. The structure of the response mirrors the structure of the query document, containing only the fields specified with `"*"` in the query document, shaped according to the query.

4.1 Status Codes

Status Code	Meaning
`200 OK`	The query was successful and the response body contains the result
`400 Bad Request`	The query document is malformed or contains invalid operators
`404 Not Found`	The resource does not exist
`405 Method Not Allowed`	The resource does not support the QUERY method
`422 Unprocessable Entity`	The query document is valid JSON but cannot be executed against this resource
`501 Not Implemented`	The server does not support the QUERY method

4.2 Response Body

The response body contains the matched data shaped according to the query document. For a single document resource the response is a JSON object. For a collection resource the response is a JSON array of objects each shaped according to the query document.

4.3 Example

Query:

```json
{
  "user": {
    "id": 1,
    "name": "*",
    "email": "*",
    "orders": {
      "status": "pending",
      "total": { ">": 100 },
      "id": "*",
      "total": "*"
    }
  }
}
```

Response:

```json
{
  "user": {
    "name": "Alice",
    "email": "alice@example.com",
    "orders": [
      { "id": 42, "total": 150 },
      { "id": 43, "total": 200 }
    ]
  }
}
```

4.4 Empty Results

A query that matches no documents returns an empty array `[]` for a collection resource or an empty object `{}` for a single document resource. An empty result is not an error and returns a `200 OK` status.

---

5. Error Semantics

Errors are returned as a JSON document with a consistent structure. The error document contains a code, a human readable message, and where applicable a detail field identifying the specific cause.

5.1 Error Document Structure

```json
{
  "error": {
    "code": "*",
    "message": "*",
    "detail": "*"
  }
}
```

5.2 Error Codes

Code	Meaning
`malformed_document`	The request body is not valid JSON
`invalid_operator`	An unrecognized operator was used
`invalid_field`	A field in the query document does not exist on the resource
`invalid_value`	A field value is of the wrong type for the operator applied to it
`unsupported_query`	The query is valid but the server does not support executing it against this resource
`resource_not_found`	The resource identified by the URL does not exist

5.3 Example

A query referencing a field that does not exist on the resource:

```json
{
  "error": {
    "code": "invalid_field",
    "message": "The query references a field that does not exist on this resource",
    "detail": "user.nonexistent_field"
  }
}
```

5.4 Design Notes

Error documents are intentionally simple. The code field is machine readable and suitable for programmatic error handling. The message field is human readable and suitable for logging and debugging. The detail field is optional and provides additional context where useful.

Servers should not expose internal implementation details in error responses.